MACHINE LEARNING LABORATORY - OBSERVATION

1. Experiment No.: 3	**Date:** ___29 /10/26 _____

2. Experiment Question

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

3. Aim

To build a decision tree with the ID3 (Iterative Dichotomiser 3) algorithm on the PlayTennis data set using Entropy and Information Gain, and to use the tree to classify a new, unseen sample.

4. Concept

A decision tree is a supervised learning model that classifies a sample by asking a sequence of questions about its attributes. Each internal node tests one attribute, each branch is one value of that attribute, and each leaf gives a class label.

ID3 builds the tree top-down with a greedy strategy. At every node it picks the attribute that gives the highest Information Gain, which is the attribute that reduces the uncertainty (entropy) of the class label the most. The data is then split on that attribute and the process repeats on each subset until a subset is pure (all one class) or no attributes remain.

- Entropy measures impurity: it is 0 for a pure set and 1 for a 50:50 split of two classes.
- Information Gain is the expected drop in entropy after splitting on an attribute.
- ID3 handles categorical attributes and does not backtrack once an attribute is chosen.

5. Dataset Description

The PlayTennis data set has 14 samples, 4 categorical input attributes and one target attribute (PlayTennis = Yes / No). It contains 9 Yes and 5 No samples.

Attribute	Possible values	Role
Outlook	Sunny, Overcast, Rain	Input
Temperature	Hot, Mild, Cool	Input
Humidity	High, Normal	Input
Wind	Weak, Strong	Input
PlayTennis	Yes, No	Target class

Day	Outlook	Temperature	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes

Day	Outlook	Temperature	Humidity	Wind	Play
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

6. Algorithm

1. Input: training set S, attribute list A, target attribute.
2. Create a root node. If all samples in S belong to one class, return a leaf with that class.
3. If A is empty, return a leaf labelled with the majority class of S.
4. Compute Gain(S, a) for every attribute a in A.
5. Select the attribute with the highest gain as the decision attribute of the node.
6. For each value v of the chosen attribute, form the subset Sv and grow a branch. Recurse with Sv and A minus the chosen attribute.
7. To classify a sample, start at the root, follow the branch that matches the sample's attribute value at every node, and output the leaf label.

7. Step-by-step Procedure

1. Load the 14 training samples and separate the attributes from the target.
2. Compute the entropy of the whole data set: 9 Yes and 5 No give E(S) = 0.9403.
3. Compute the Information Gain of Outlook, Temperature, Humidity and Wind. Outlook has the highest gain (0.2467), so it becomes the root.
4. Split on Outlook. The Overcast branch is pure (all Yes) and becomes a leaf.
5. For Sunny, Humidity gives the best gain: High leads to No and Normal leads to Yes.
6. For Rain, Wind gives the best gain: Weak leads to Yes and Strong leads to No.
7. Classify the new sample (Sunny, Cool, High, Strong) by traversing the tree: Outlook = Sunny, then Humidity = High, so the class is No.

8. Formulae

Entropy of a set S with c classes

$$E(S) = - SUM[\, p_i * log2(p_i) \,], \quad i = 1 \dots c$$

Information Gain of attribute A

$$Gain(S, A) = E(S) - SUM[\ (|Sv| / |S|) * E(Sv)\ ],\quad v\ in\ Values(A)$$

Worked root-node values

$$E(S) = -(9/14)log2(9/14) - (5/14)log2(5/14) = 0.9403$$

$$Gain(S, Outlook) = 0.9403 - [(5/14)(0.9710) + (4/14)(0) + (5/14)(0.9710)] = 0.2467$$

9. Python Program

```
"""
Experiment 3: Decision Tree using the ID3 Algorithm
Dataset : PlayTennis (14 samples, Quinlan)
"""
import math
from collections import Counter
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- dataset
FEATURES = ["Outlook", "Temperature", "Humidity", "Wind"]
TARGET = "PlayTennis"
DATA = [
    ("Sunny",    "Hot",  "High",   "Weak",   "No"),
    ("Sunny",    "Hot",  "High",   "Strong", "No"),
    ("Overcast", "Hot",  "High",   "Weak",   "Yes"),
    ("Rain",     "Mild", "High",   "Weak",   "Yes"),
    ("Rain",     "Cool", "Normal", "Weak",   "Yes"),
    ("Rain",     "Cool", "Normal", "Strong", "No"),
    ("Overcast", "Cool", "Normal", "Strong", "Yes"),
    ("Sunny",    "Mild", "High",   "Weak",   "No"),
    ("Sunny",    "Cool", "Normal", "Weak",   "Yes"),
    ("Rain",     "Mild", "Normal", "Weak",   "Yes"),
    ("Sunny",    "Mild", "Normal", "Strong", "Yes"),
    ("Overcast", "Mild", "High",   "Strong", "Yes"),
    ("Overcast", "Hot",  "Normal", "Weak",   "Yes"),
    ("Rain",     "Mild", "High",   "Strong", "No"),
]
rows = [dict(zip(FEATURES + [TARGET], r)) for r in DATA]

# ---------------------------------------------------------------- ID3 core
def entropy(data):
    """E(S) = - sum p_i log2 p_i"""
    total = len(data)
    counts = Counter(r[TARGET] for r in data)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())


def info_gain(data, attr):
    """Gain(S, A) = E(S) - sum (|Sv|/|S|) E(Sv)"""
    total = len(data)
    remainder = 0.0
    for value in set(r[attr] for r in data):
        subset = [r for r in data if r[attr] == value]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(data) - remainder


def id3(data, attrs):
    labels = [r[TARGET] for r in data]
    if len(set(labels)) == 1:                 # pure node -> leaf
        return labels[0]
    if not attrs:                             # no attributes left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda a: info_gain(data, a))
    tree = {best: {}}
    for value in sorted(set(r[best] for r in data)):
        subset = [r for r in data if r[best] == value]
        tree[best][value] = id3(subset, [a for a in attrs if a != best])
    return tree


def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree


def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "-> " + str(tree)); return
    attr = next(iter(tree))
    for value, sub in tree[attr].items():
        if isinstance(sub, dict):
            print(f"{indent}[{attr} = {value}]")
            print_tree(sub, indent + "    ")
        else:
            print(f"{indent}[{attr} = {value}] -> {sub}")


# ---------------------------------------------------------------- drawing
def draw_tree(tree, fname):
    """Greyscale drawing of the decision tree."""
    positions, edges = {}, []
    counter = [0]
    def layout(node, depth):
        if not isinstance(node, dict):
```

```
            x = counter[0]; counter[0] += 1
            return x, ("leaf", str(node), depth, x)
        attr = next(iter(node)); xs = []; kids = []
        for v, sub in node[attr].items():
            cx, info = layout(sub, depth + 1)
            xs.append(cx); kids.append((v, info))
        x = sum(xs) / len(xs)
        return x, ("node", attr, depth, x, kids)
    _, root = layout(tree, 0)
    fig, ax = plt.subplots(figsize=(9, 5))
    def draw(info):
        kind, label, depth, x = info[:4]
        y = -depth
        if kind == "node":
            ax.text(x, y, label, ha="center", va="center", fontsize=10, fontweight="bold",
                    bbox=dict(boxstyle="round,pad=0.4", fc="#e6e6e6", ec="black"))
            for v, kid in info[4]:
                ky = -kid[2]
                ax.plot([x, kid[3]], [y - 0.12, ky + 0.12], color="black", lw=1)
                ax.text((x + kid[3]) / 2, (y + ky) / 2 + 0.03, v, ha="center", fontsize=8,
                        bbox=dict(fc="white", ec="none", pad=0.5))
                draw(kid)
        else:
            ax.text(x, y, label, ha="center", va="center", fontsize=10,
                    bbox=dict(boxstyle="square,pad=0.4", fc="white", ec="black", lw=1.4))
    draw(root)
    ax.axis("off"); ax.set_title("ID3 Decision Tree - PlayTennis", fontsize=11)
    plt.tight_layout(); plt.savefig(fname, dpi=200); plt.close()


# ---------------------------------------------------------------- main
if __name__ == "__main__":
    print("Entropy of the full data set E(S) = %.4f" % entropy(rows))
    print("\nInformation Gain at the root node:")
    for a in FEATURES:
        print(f"  Gain(S, {a:<12}) = {info_gain(rows, a):.4f}")


    tree = id3(rows, FEATURES)
    print("\nDecision tree built by ID3:")
    print_tree(tree)


    new_sample = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "High", "Wind": "Strong"}
    print("\nNew sample :", new_sample)
    print("Prediction : PlayTennis =", classify(tree, new_sample))


    correct = sum(classify(tree, r) == r[TARGET] for r in rows)
    print(f"\nTraining accuracy: {correct}/{len(rows)} = {100*correct/len(rows):.1f}%")
    draw_tree(tree, "figs/exp3_tree.png")
```

10. Output

```
Entropy of the full data set E(S) = 0.9403


Information Gain at the root node:
  Gain(S, Outlook     ) = 0.2467
  Gain(S, Temperature ) = 0.0292
  Gain(S, Humidity    ) = 0.1518
  Gain(S, Wind        ) = 0.0481


Decision tree built by ID3:
[Outlook = Overcast] -> Yes
[Outlook = Rain]
    [Wind = Strong] -> No
    [Wind = Weak] -> Yes
[Outlook = Sunny]
    [Humidity = High] -> No
    [Humidity = Normal] -> Yes


New sample : {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Prediction : PlayTennis = No


Training accuracy: 14/14 = 100.0%
```

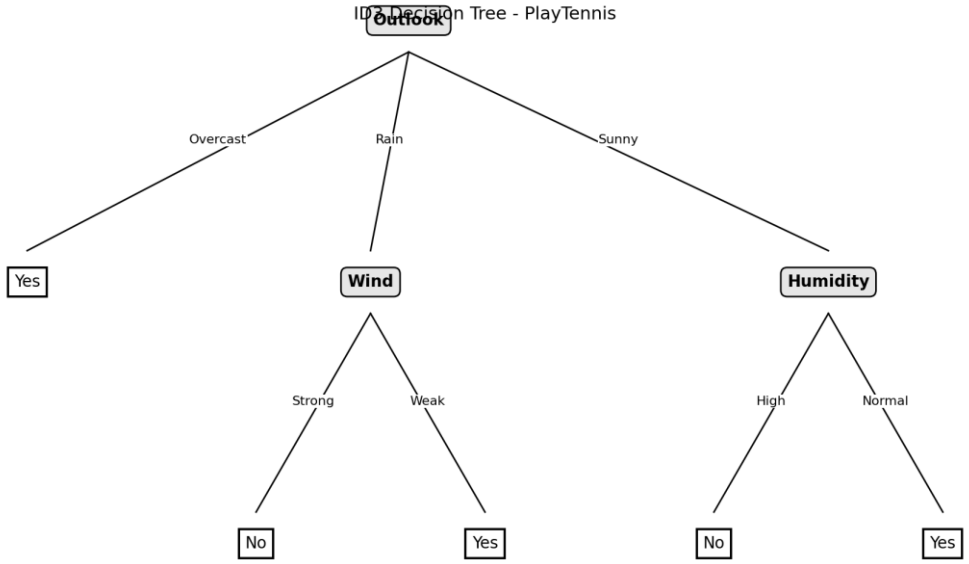


Fig. 3.1 Decision tree generated by ID3

11. Result / Conclusion

The ID3 algorithm built a decision tree with Outlook at the root, Humidity under Sunny and Wind under Rain. The tree classifies all 14 training samples correctly. The new sample (Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong) was classified as PlayTennis = No. Hence the working of the ID3 algorithm was demonstrated successfully.